

Generative Models

Soo Min Kimm

07/11/25

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

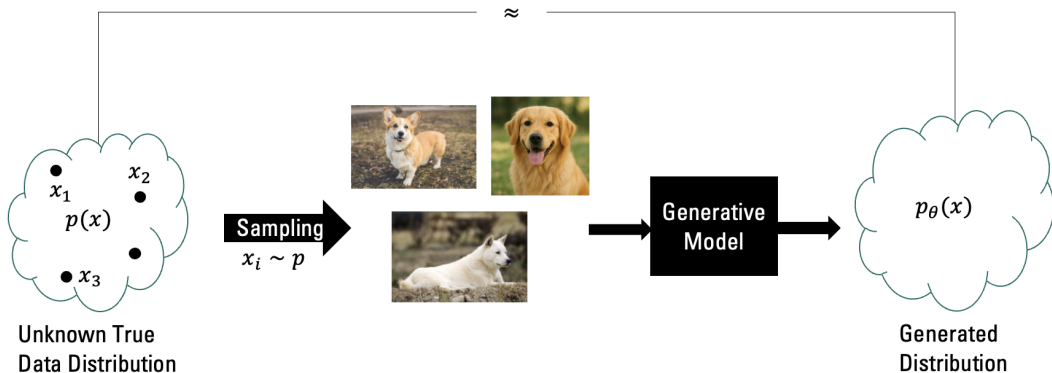


Figure: Generative Model

Constraints

- Non-negativity:

$$p_{\theta}(x) \geq 0 \quad \forall x$$

- Normalisation:

$$\int p_{\theta}(x) dx = 1$$

KL Divergence

Given $p(x)$, the true distribution the data actually follows, and $p_\theta(x)$, the approximate distribution,

$$D_{\text{KL}}(p \| p_\theta) = \mathbb{E}_{x \sim p(x)} \left[\log \frac{p(x)}{p_\theta(x)} \right] = \int p(x) \log \frac{p(x)}{p_\theta(x)} dx = H(p, p_\theta) - H(p) \geq 0$$

measures the closeness of two distributions.

Training Objective

Training objective of the generative model is to find θ that minimises KL divergence.
As $p(x)$ is not dependent on θ ,

$$\hat{\theta} = \arg \min_{\theta} D_{\text{KL}}(p \| p_{\theta}) = \arg \max_{\theta} \mathbb{E}_{x \sim p(x)} [\log p_{\theta}(x)]$$

which is equivalent to maximum likelihood estimation (MLE).

Loss function is defined as

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim p(x)} [\log p_{\theta}(x)]$$

Taxonomy

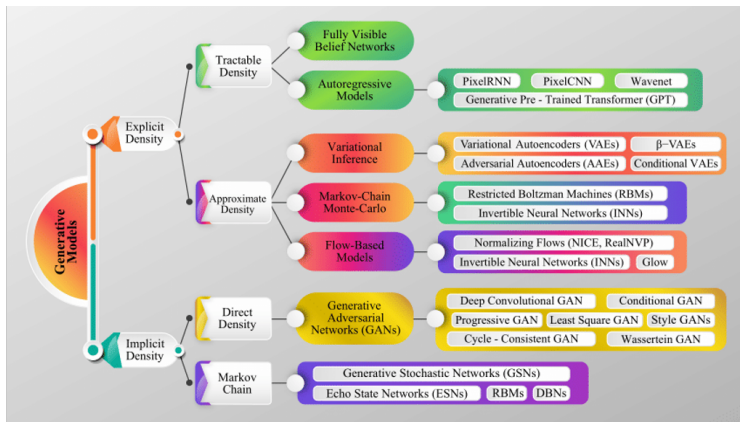


Figure: Taxonomy. Image from Çelik & Eltawil (2024) – At the Dawn of Generative AI Era.

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

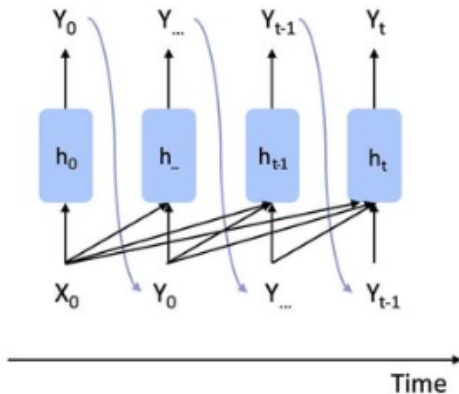


Figure: Autoregressive model. Image from Mardikoraem et al. (2023) - Generative Models for Protein Sequence Modelling.

Objective

ARM factorises the joint distribution using the chain rule

$$p_{\theta}(x) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}) \quad \checkmark \text{Tractable}$$

for sequence $x = (x_1, \dots, x_T)$.

Loss function is

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}) \quad \checkmark \text{Tractable}$$

Architecture

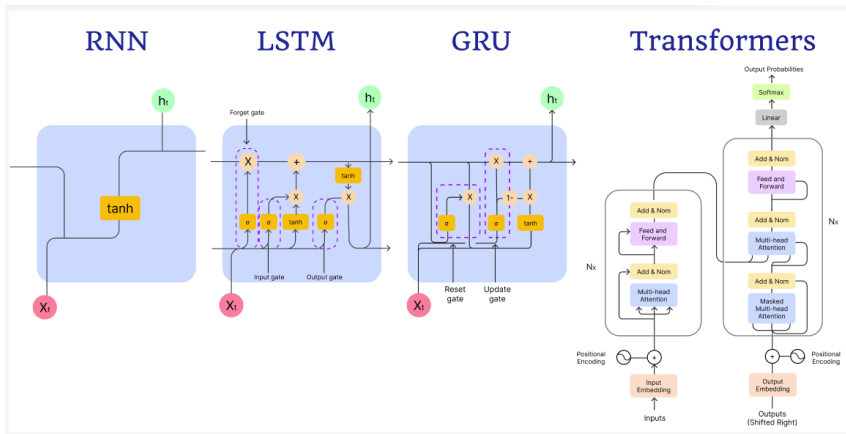


Figure: Examples of AR models. Image from <https://code-b.dev/blog/autoregressive-language-model>.

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- **Variational Autoencoder (VAE)**
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

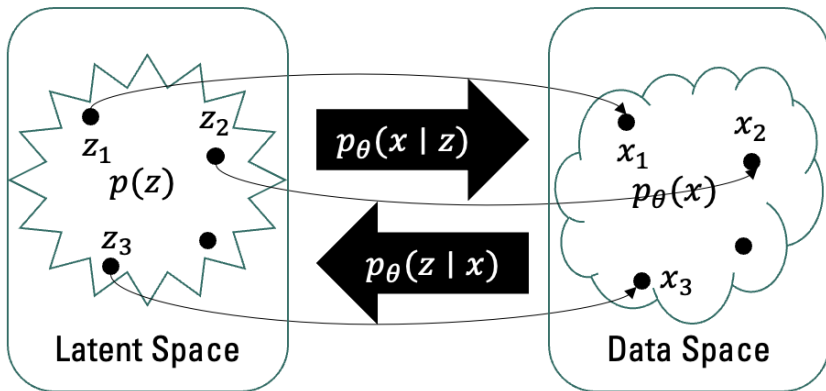


Figure: Latent & Data Space

Definition

Joint distribution

$$p(x, z) = p(x | z)p(z)$$

Marginal likelihood (Evidence)

$$p(x) = \int p(x, z)dz$$

Posterior

$$p(z | x) = \frac{p(z)p(x|z)}{p(x)}$$

Jensen's Inequality

Let f be a convex function and X a random variable, then $f(\mathbb{E}[X]) \leq \mathbb{E}[f(x)]$.
Inequality reverses for the concave function.

Assumptions

- Prior:
Set $p(z) = \mathcal{N}(0, I)$ ✓ Tractable to make sampling and regularisation easy.
- Likelihood:
Set $p_\theta(x | z) = \mathcal{N}(\mu_\theta(z), \sigma_\theta^2(z)I)$ ✓ Tractable parameterised by the decoder.
- Posterior:
 $p_\theta(z | x)$ ✗ Intractable is intractable.
Introduce an approximate posterior
 $q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)I)$ ★ Approx./Learned parameterised by the encoder.

Evidence Lower Bound (ELBO)

$$\log p_{\theta}(x) = \log \int p_{\theta}(x, z) dz$$

(Definition of marginal likelihood)

$$= \log \int p_{\theta}(x, z) \frac{q_{\phi}(z | x)}{q_{\phi}(z | x)} dz$$

$$= \log \mathbb{E}_{q_{\phi}(z|x)} \frac{p_{\theta}(x, z)}{q_{\phi}(z | x)}$$

(Definition of expectation)

$$\geq \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x | z) p(z)}{q_{\phi}(z | x)} \right]$$

(Jensen's inequality)

$$= \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)]}_{\mathcal{L}_{\text{rec}} \quad \star \text{Approx./Learned}} - \underbrace{D_{KL}(q_{\phi}(z | x) \| p(z))}_{\mathcal{L}_{\text{reg}} \quad \checkmark \text{Tractable}}$$

(Definition of KL Divergence)

Monte Carlo Method

Monte Carlo method is a way to approximate expectations (integrals) using random samples.

Integral is often too complicated to compute exactly.

Instead, draw samples $x^{(1)}, \dots, x^{(N)} \sim p(x)$.

If N is large enough, by the law of large numbers,

$$\mathbb{E}_{x \sim p(x)}[f(x)] = \int f(x)p(x)dx \approx \frac{1}{N} \sum_{i=1}^N f(x^{(i)})$$

Objective

The objective of VAE is equivalent to maximising the ELBO.

By the Monte-Carlo approximation,

$$\mathbb{E}_{z \sim q_{\phi}(z|x)}[\log f(z)] \approx \frac{1}{L} \sum_{l=1}^L f(z^{(l)}) \quad \checkmark \text{Tractable}$$

where $z^{(l)} \sim q_{\phi}(z | x)$ and $f(z) = \log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)}$.

Reparameterisation Trick

Samples from a Gaussian distribution $z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)I)$ can be rewritten as a combination of a deterministic function and a noise variable

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

where $\epsilon \sim \mathcal{N}(0, 1)$.

Architecture

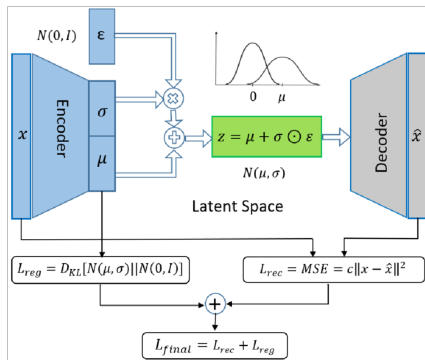


Figure: VAE. Image from Puchalski & Komorska (2023) – Generative Modelling of Vibration Signals in Machine Maintenance.

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- **Normalising Flow**
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

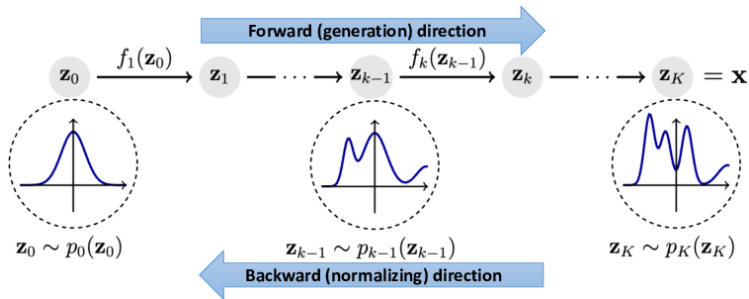


Figure: Normalising flow. Image from Shen & Shen (2023) – PSRFlo.

Flow (Discrete)

Let f_θ be invertible, differential map where $z = f_\theta^{-1}(x)$.

$f_\theta = f_K \circ f_{K-1} \circ \cdots \circ f_1$ where f_k is chosen such that the Jacobian determinant is cheap.

Change of Variables Theorem

Suppose $z \in \mathbb{R}^d$, and $x = f(z)$ with $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ a diffeomorphism.

$$p_X(x) = p_Z(f_\theta^{-1}(x)) |\det J_{f_\theta^{-1}}(x)| = p_Z(z) \left| \frac{1}{\det J_{f_\theta}(z)} \right|$$

Objective

By the change of variables theorem,

$$\log p_X(x) = \log p_Z(f_\theta^{-1}(x)) - \log |\det J_{f_\theta}(f_\theta^{-1}(x))|$$

Since $f_\theta = f_K \circ f_{K-1} \circ \dots \circ f_1$, starting with the data space density $p_{Z_0}(z_0) = p_X(x)$, the first transformation $f_1 : z_0 \mapsto z_1$ gives $\log p_{Z_0}(z_0) = \log p_{Z_1}(z_1) - \log |\det J_{f_1}(z_0)|$.

The second transformation $f_2 : z_1 \mapsto z_2$ gives

$$\log p_{Z_1}(z_1) = \log p_{Z_2}(z_2) - \log |\det J_{f_2}(z_1)|.$$

After repeating for all K transformations with $z_0 = x$ and $z_k = z$,

$$\log p_X(x) = \log p_Z(z) - \sum_{k=1}^K \log |\det J_{f_k}(z_{k-1})| \quad \checkmark \text{ Tractable}$$

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- **Flow Matching**
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

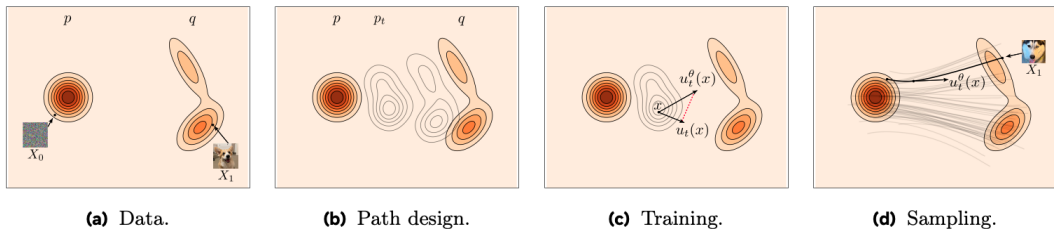


Figure: Flow matching. Image from Lipman et al. (2024) - Flow Matching Guide and Code.

Flow (Continuous)

ODE $\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x))$ where $\psi_t = \psi(t, x)$ and $\psi_0(x) = x$ describes how a point in space moves over time under a velocity rule.

$\psi_t \in \mathbb{R}^d$ is a solution to the ODE called a trajectory, which represents the position of a particle at time t .

$u_t(x) \in \mathbb{R}^d$ is a vector field that gives the velocity at location x at time t .

Definition

Probability Path

The velocity field u_t generates the probability path p_t if its flow ψ_t satisfies $X_t = \psi_t(X_0) \sim p_t$ for $X_0 \sim p_0$.

Push Forward

ODE induces a push-forward of distributions $p_t = (\psi_t)_\# p_0$ meaning if sample $x_0 \sim p_0$ is pushed through the flow ψ_t , the resulting distribution is p_t .

By the change of variables theorem, if $y = \psi_t(x)$, then

$$p_t(y) = p_0(\psi_t^{-1}(y)) \left| \det \frac{\partial}{\partial y} \psi_t^{-1}(y) \right|.$$

Continuity Equation

The evolution of probability densities under a velocity field is governed by the continuity equation $\frac{\partial}{\partial t} p_t(x) + \nabla \cdot (p_t(x) u_t(x)) = 0$.

Assumptions

True marginal velocity field $u_t(x)$ **✗ Intractable** that takes $p_0 \rightarrow p_1$ is intractable.

Construct an interpolation between samples of p_0 and p_1 .

For two endpoints $x_0 \sim p_0$ and $x_1 \sim p_1$, using a linear interpolation,

$$x_t = (1 - t)x_0 + tx_1.$$

If x_0, x_1 are fixed, then x_t is deterministic.

Hence, the random variable X_t conditioned on a pair (x_0, x_1) has distribution

$$p_t(x \mid x_0, x_1) = \delta(x - ((1 - t)x_0 + tx_1)).$$

Furthermore, $\frac{d}{dt}x_t = x_1 - x_0 = \frac{x_1 - x_t}{1 - t} = u_t^*(x_t \mid x_1)$ **✓ Tractable** becomes a tractable conditional target velocity.

Objective

The objective of flow matching is to train a model $u_\theta(x, t)$ by minimising MSE loss

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}[\|u_\theta(x_t, t) - u_t(x_t)\|^2] \quad \text{✗ Intractable}$$

As the true marginal velocity field is intractable, use the conditional target velocity field instead, i.e.

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}[\|u_\theta(x_t, t) - u_t^*(x_t \mid x_1)\|^2] \quad \text{✓ Tractable}$$

Using conditional target velocity is valid because optimisation with respect to θ yields $\nabla_\theta \mathcal{L}_{\text{FM}} = \nabla_\theta \mathcal{L}_{\text{CFM}}$.

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- **Energy-Based Model (EBM)**
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

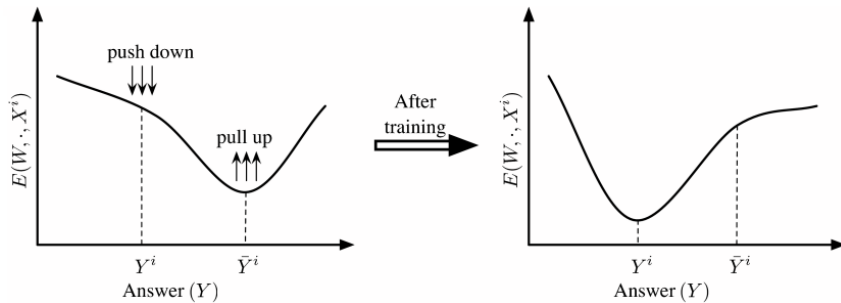


Figure: Energy-based model.

Objective

EBM defines a probability distribution

$$p_{\theta}(x) = \frac{\exp(-E_{\theta}(x))}{Z(\theta)} \quad \text{✗ Intractable}$$

where $E_{\theta}(x)$ ✓ Tractable is the energy function and

$Z(\theta) = \int \exp(-E_{\theta}(x))dx$ ✗ Intractable is the partition function for normalisation.

$\log p_{\theta}(x) = -E_{\theta}(x) - \log Z(\theta)$ but $Z(\theta)$ is intractable.

By taking gradients with respect to θ , $\nabla_{\theta} \log p_{\theta}(x) = -\nabla_{\theta} E_{\theta}(x) - \nabla_{\theta} \log Z(\theta)$

Partition Function Trick

Differentiating $Z(\theta)$ yields $\nabla_{\theta} Z(\theta) = \int \nabla_{\theta} \exp(-E_{\theta}(x)) dx$.

By the chain rule, $\nabla_{\theta} \exp(-E_{\theta}(x)) = (-\nabla_{\theta} E_{\theta}(x)) \exp(-E_{\theta}(x))$

Hence, $\nabla_{\theta} Z(\theta) = \int (-\nabla_{\theta} E_{\theta}(x)) \exp(-E_{\theta}(x)) dx$.

Dividing by $Z(\theta)$ gives

$$\begin{aligned}\frac{\nabla_{\theta} Z(\theta)}{Z(\theta)} &= \nabla_{\theta} \log Z(\theta) \\ &= \int (-\nabla_{\theta} E_{\theta}(x)) \frac{\exp(-E_{\theta}(x))}{Z(\theta)} dx \\ &= \int (-\nabla_{\theta} E_{\theta}(x)) p_{\theta}(x) dx \\ &= -\mathbb{E}_{x \sim p_{\theta}(x)} [\nabla_{\theta} E_{\theta}(x)] \quad \star \text{Approx./Learned}\end{aligned}$$

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- **Diffusion**
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

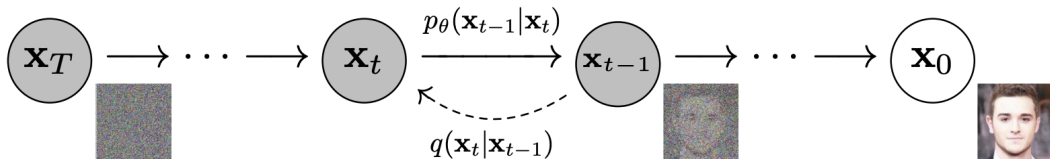


Figure: Diffusion. Image from Ho et al. (2020) - DDPM.

Probability Chain Rule

For any sequence of random variables $x_0, \dots, x_T$, the joint distribution factorises as

$$p(x_0, \dots, x_T) = p(x_0)p(x_1 | x_0)p(x_2 | x_0, x_1) \cdots p(x_T | x_{0:T-1})$$

A process is Markov chain if the future depends only on the present, not the past:

$$p(x_t | x_0, x_1, \dots, x_{t-1}) = p(x_t | x_{t-1})$$

Hence, the joint distribution reduces to

$$p(x_{0:T}) = p(x_0) \prod_{t=1}^T p(x_t | x_{t-1})$$

if it is Markovian.

Forward Process

By construction, the diffusion process follows the Markovian Gaussian distribution:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I) \quad \checkmark \text{Tractable}$$

where $\alpha_t = 1 - \beta_t \in [0, 1]$.

Using the reparameterisation trick, $x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t} \cdot \epsilon_t$ where $\epsilon_t \sim \mathcal{N}(0, 1)$.

Recursively,

$$\begin{aligned} x_t &= \sqrt{\alpha_1 \cdots \alpha_t}x_0 + \sum_{s=1}^t ((\sqrt{1 - \alpha_s} \prod_{j=s+1}^t \sqrt{\alpha_j}) \cdot \epsilon_s) \\ &= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon \end{aligned}$$

where $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ and $\epsilon \sim \mathcal{N}(0, 1)$.

This implies

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad \checkmark \text{Tractable} \xrightarrow{t \rightarrow \infty} \mathcal{N}(0, I)$$

Reverse Process

True posterior can be defined as

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I) \quad \checkmark \text{Tractable}$$

where $\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}}\epsilon)$ with $\epsilon = \frac{x_t - \sqrt{\alpha_t}x_0}{\sqrt{1-\alpha_t}}$ and $\tilde{\beta}_t = \frac{1-\alpha_{t-1}}{1-\alpha_t}\beta_t$.

During the inference, x_0 is not accessible; hence, choose a Gaussian distribution

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad \star \text{Approx./Learned}$$

such that $\mu_\theta, \Sigma_\theta$ match the true posterior.

This is called a reverse kernel.

Complete Process

Complete forward and reverse processes can be obtained using the probability chain rule.

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1})$$

where $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I)$, and

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t)$$

where $p(x_T) = \mathcal{N}(0, I)$.

Objective

$$\begin{aligned}
 \log p_{\theta}(x_0) &= \log \int p_{\theta}(x_{0:T}) dx_{1:T} \\
 &\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} \right] \\
 &= \underbrace{\mathbb{E}_{q(x_1|x_0)} [\log p_{\theta}(x_0 | x_1)]}_{\text{Reconstruction term} \quad \star \text{Approx./Learned}} - \underbrace{D_{\text{KL}}(q(x_T | x_0) \| p(x_T))}_{\text{Prior matching term}} \\
 &\quad - \underbrace{\sum_{t=2}^T \mathbb{E}_{q(x_{1:T}|x_0)} [D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_{\theta}(x_{t-1} | x_t))]}_{\text{Denoising matching term} \quad \checkmark \text{Tractable}}
 \end{aligned}$$

Simplified Objective

KL Divergence between two Gaussian distributions is

$$D_{\text{KL}}(\mathcal{N}(\mu_x, \Sigma_x) \parallel \mathcal{N}(\mu_y, \Sigma_y)) = \frac{1}{2} \left[\log \frac{|\Sigma_y|}{|\Sigma_x|} - d + \text{tr}(\Sigma_y^{-1} \Sigma_x) + (\mu_y - \mu_x)^T \Sigma_y^{-1} (\mu_y - \mu_x) \right]$$

KL term reduces to expectations of squared errors between true $\tilde{\mu}$ and predicted means μ_θ .

Simplified loss is given by

$$\mathcal{L}(\theta) = \mathbb{E} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right] \quad \checkmark \text{Tractable}$$

where $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$.

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- **Score-Based Models**
- Generative Adversarial Network (GAN)

3 Summary

Intuition

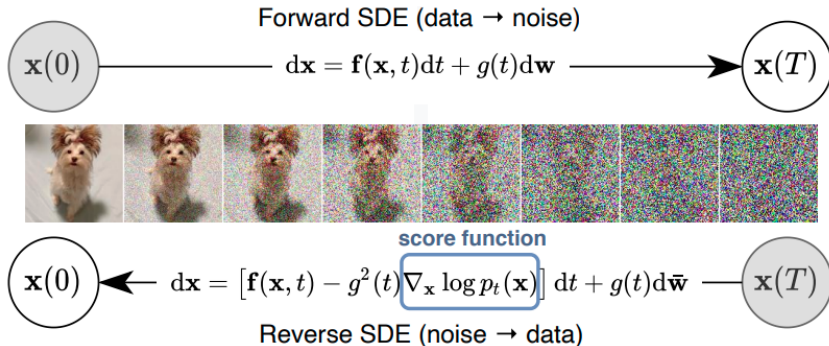


Figure: Score matching. Image from Song et al. (2020) – Score-Based Generative Model through SDE.

Score Function

For a probability density $p(x)$, the score function is

$$s(x) = \nabla_x \log p(x)$$

which tells which direction in data space increases probability the fastest.

At the mode of the distribution, the score is zero.

For the case of EBM, $s(x) = -\nabla_x E(x)$, which removes the necessity of computing $Z(\theta)$.

Tweedie's Formula

Suppose the data is corrupted by Gaussian noise, i.e. $x = x_0 + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$. Then the Tweedie's formula says

$$\mathbb{E}[x_0 \mid x] = x + \sigma^2 \nabla_x \log p_\sigma(x)$$

where $p_\sigma(x)$ is the marginal distribution of noisy observations.

Posterior mean is given by the noisy point x and a correction from the score.

Since the posterior mean is the best possible estimate of x_0 given noisy observation x , denoising is equivalent to estimating a score function.

Fisher Divergence

Just like KL divergence, the Fisher divergence

$$D_F(p||p_\theta) = \mathbb{E}_{x \sim p(x)} [\|\nabla_x \log p(x) - \nabla_x \log p_\theta(x)\|^2]$$

measures the closeness of two score functions.

Objective

Directly minimising D_F requires the true score $s(x)$ **✗ Intractable**, which is unknown. Hyvärinen (2005) showed an equivalent, tractable objective:

$$J(\theta) = \mathbb{E}_{x \sim p(x)} \left[\frac{1}{2} \|s_\theta(x)\|^2 + \text{tr}(\nabla_x s_\theta(x)) \right] \quad \checkmark \text{Tractable}$$

This only uses data samples $x \sim p(x)$ and the model score $s_\theta(x)$.

Forward Process

Score-based models describe data noising by an Itô SDE:

$$dx = f(x, t) dt + g(t) dW_t$$

where $f(x, t)$ is the drift term (deterministic), $g(t)$ is the diffusion strength, and W_t is the Wiener process or Brownian motion (stochastic).

- Variance-preserving (VP) SDE:

$$dx = -\frac{1}{2}\beta(t)x dt + \sqrt{\beta(t)} dW_t \implies x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon \text{ with } \alpha_t = \exp\left(-\int_0^t \beta(s)ds\right)$$

- Variance-exploding (VE) SDE:

$$dx = g(t) dW_t \implies x_t = x_0 + \sigma(t)\epsilon \text{ with } \sigma(t) = \int_0^t g^2(s)ds$$

Reverse Process

Anderson (1982) proved that if $\{x_t\}_{t \in [0, T]}$ follows the forward SDE with marginal distribution $p(x)$, the time reversed process is also an SDE:

$$dx = [f(x, t) - g^2(t) s(x, t)]dt + g(t) d\bar{W}_t$$

where $\bar{W}_t$ is a Brownian motion in reverse time.

Langevin Dynamics

Langevin dynamics provides how particles move in a fluid, where they feel both deterministic forces and random collisions.

The general Langevin equation for a particle with mass m is given as

$$m \frac{d^2 x}{dt^2} = -\gamma \frac{dx}{dt} - \nabla U(x) + \sqrt{2\gamma k_B T} \xi(t)$$

where $-\gamma \dot{x}$ is the friction or damping term, $-\nabla U(x)$ is the deterministic force from potential energy $U(x)$, and $\sqrt{2\gamma k_B T} \xi(t)$ is the thermal noise.

Overdamped Langevin dynamics in terms of probability density $p(x) \propto e^{-U(x)}$ can be written as

$$dx = \nabla_x \log p(x) dt + \sqrt{2} dW_t$$

Discretised case for sampling is given by Euler-Maruyama equation

$$x_{k+1} = x_k + \eta \nabla_x \log p(x_k) + \sqrt{2\eta} z_k$$

with $z_k \sim \mathcal{N}(0, I)$ where η is the step size.

Predictor-Corrector Samplers

Direct discretisation can produce unstable samples.

The predictor-corrector method improves stability by alternating

- Predictor step: Move sample along reverse SDE (Global trajectory)
- Corrector step: Refine sample with Langevin dynamics (Local adjustment)

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Intuition

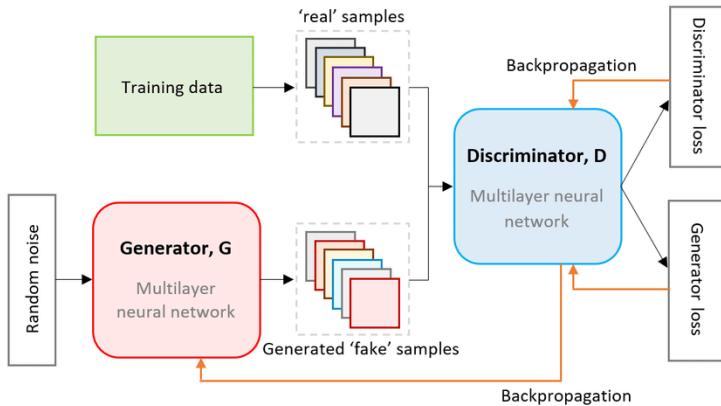


Figure: GAN. Image from Little et al. (2021) – Generative Adversarial Networks for Synthetic Data Generation.

Minimax Game

- Generator $G(z)$: Map noise $z \sim p(z)$ to fake samples, inducing distribution $p_g(x)$ **✗ Intractable**.
- Discriminator $D(x)$: Output probability that x is real
- Value function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p(x)} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))] \quad \text{✓ Tractable}$$

Objective

Binary cross entropy is given by $BCE(y, D(x)) = -y \log D(x) - (1 - y) \log (1 - D(x))$. If the sample is real ($y = 1$), the loss is $-\log D(x)$ and if it is fake ($y = 0$), the loss is $-\log (1 - D(x))$.

Discriminator's objective is

$$\mathcal{L}(D) = -\mathbb{E}_{x \sim p(x)}[\log D(x)] - \mathbb{E}_{z \sim p(z)}[\log (1 - D(G(z)))]$$

Generator's objective is

$$\mathcal{L}(G) = \mathbb{E}_{z \sim p(z)}[\log (1 - D(G(z)))]$$

Note that when $D(G(z)) \rightarrow 0$, the gradient vanishes; flip the label to 1 for fake samples, i.e.,

$$\mathcal{L}(G) = -\mathbb{E}_{z \sim p(z)}[\log D(G(z))]$$

Optimal Solution

Setting $p_g(x)$ fixed, then

$$f(x) = p(x) \log D(x) + p_g(x) \log (1 - D(x))$$

$$\frac{\partial f}{\partial D} = \frac{p(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} = 0 \iff D^*(x) = \frac{p(x)}{p(x) + p_g(x)}$$

where D^* is the optimal discriminator.

Introduce a mixture distribution $M(x) = \frac{1}{2}(p(x) + p_g(x))$, then the value function reduces to

$$\begin{aligned} V(D^*, G) &= \mathbb{E}_p \left[\log \frac{p(x)}{M(x)} \right] + \mathbb{E}_{p_g} \left[\log \frac{p_g(x)}{M(x)} \right] - 2 \log 2 \\ &= D_{\text{KL}}(p(x) \| M(x)) + D_{\text{KL}}(p_g(x) \| M(x)) - 2 \log 2 \\ &= -\log 4 + 2D_{\text{JS}}(p(x) \| p_g(x)) \end{aligned}$$

where $D_{\text{JS}}(p \| p_g) = \frac{1}{2} D_{\text{KL}}(p \| M) + \frac{1}{2} D_{\text{KL}}(p_g \| M)$ is the Jensen-Shannon divergence.

At the global optimum, $p_g(x) = p(x)$, $D_{\text{JS}} = 0$, and $V(D^*, G) = -\log 4$.

Outline

1 Motivation

2 Models

- Autoregressive Model (ARM)
- Variational Autoencoder (VAE)
- Normalising Flow
- Flow Matching
- Energy-Based Model (EBM)
- Diffusion
- Score-Based Models
- Generative Adversarial Network (GAN)

3 Summary

Architecture

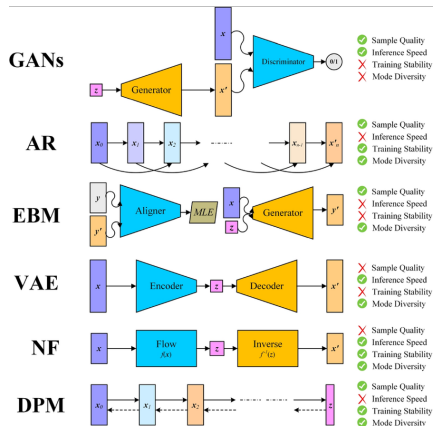


Figure: Architecture of different generative models. Image from Ahmad et al. (2024) – Understanding GANs.

Model Landscape

Family	Model	Key Idea
Explicit (factorized)	ARM	Chain-rule factorisation of joint distribution
Explicit (latent)	VAE	- Latent variables + variational inference (ELBO) - Balances reconstruction and regularisation
Explicit (invertible)	Normalising Flow	- Sequence of invertible maps with tractable Jacobians - Discrete-step explicit density model
Continuous flows	Flow Matching (ODE)	- Continuous analogue of flows via velocity fields - Generalises diffusion without discrete Markov chain
Score/Energy	EBM	- Defines unnormalised density via energy - Direct link to score: $s(x) = -\nabla E(x)$
Score/Energy	Diffusion (DDPM)	Discrete Markov noise process
Score/Energy	Score-Based (SDE)	- Learns time-dependent scores $s(x, t)$ - Reverse-time SDE unifies with EBMs/diffusion
Implicit (adversarial)	GAN	- No explicit likelihood - Adversarial game to match data distribution

Comparison of Generative Models (1/2)

Model	Pros	Cons
ARM	- Exact likelihood - Stable MLE training - Strong for sequential data	- Slow sequential sampling - Limited parallelism - Exposure bias
VAE	- Learns latent representations - Tractable ELBO - Smooth interpolation	- Variational gap - Blurry reconstructions - Posterior mismatch
Normalising Flow	- Exact log-likelihood - Fast inference - Principled density modeling	- Invertibility/Jacobian constraints - Limited expressivity on complex data
Flow Matching (ODE)	- Direct continuous path - Simple MSE training - Fewer steps than diffusion	- Requires tractable conditional velocities - Still new; fewer benchmarks

Comparison of Generative Models (2/2)

Model	Pros	Cons
EBM	• Very flexible (no invertibility/latent vars) • Natural physics analogy • Connects to scores	• Partition function intractable • Requires expensive MCMC • Training unstable at scale
Diffusion (DDPM)	• State-of-the-art fidelity • Stable training • Likelihood surrogate (ELBO)	• Very slow sampling (many steps) • Careful noise schedule required
Score-Based (SDE)	• Unifies diffusion/EBM/Langevin • Flexible SDE choices • High quality with PC samplers	• Training across many noise levels • Sampling slower than ARMs/GANs
GAN	• Sharp, high-fidelity one-shot samples • Very fast generation	• Training instability (mode collapse) • No explicit likelihood • Sensitive to hyperparameters